

Module 3: Seccomp Profiles - Assessment Questions

Module-Level Questions (3 MCQs)

Question 1: Seccomp Fundamentals

Question:

What is the primary function of Seccomp (Secure Computing Mode) in container security?

- A) To encrypt all data transmitted between containers
- B) To filter and restrict which system calls a process can make to the kernel
- C) To isolate container networks from the host network
- D) To scan container images for known vulnerabilities

Correct Answer: B

Explanation:

Seccomp (Secure Computing Mode) is a Linux kernel feature that filters system calls at the boundary between user space and kernel space. When a process attempts to make a system call, Seccomp checks it against a defined profile and either allows it, blocks it with an error, or logs it. This dramatically reduces the attack surface by preventing compromised containers from executing dangerous kernel operations.

Why other options are incorrect:

- **A** is incorrect because Seccomp does not handle encryption; it filters syscalls.
- **C** is incorrect because network isolation is handled by network namespaces and network policies, not Seccomp.
- **D** is incorrect because image vulnerability scanning is done by tools like Trivy or Gype, not Seccomp.

Learning Objective Tested: Explain how Seccomp filters system calls at the kernel level

Question 2: Custom Seccomp Profiles

Question:

You are creating a custom Seccomp profile for an nginx web server. After capturing syscalls with strace, you build a profile with 45 allowed syscalls. When you test the profile, nginx starts successfully but fails to serve HTTP requests with "Connection refused" errors. Which of the following is the MOST likely cause?

- A) The `read` and `write` syscalls are missing from the profile
- B) The `socket`, `bind`, `listen`, or `accept` syscalls are missing from the profile
- C) The `defaultAction` is set to `SCMP_ACT_ALLOW` instead of `SCMP_ACT_ERRNO`
- D) The profile includes too many syscalls, causing a conflict

Correct Answer: B

Explanation:

When nginx starts successfully but cannot serve HTTP requests, the most likely cause is missing

networking-related syscalls. The syscalls `socket`, `bind`, `listen`, and `accept / accept4` are essential for creating server sockets, binding to ports, listening for connections, and accepting incoming requests. If these are blocked, nginx can start (which uses different syscalls) but cannot establish network connections.

Why other options are incorrect:

- **A** is incorrect because while `read` and `write` are needed for I/O, their absence would cause different errors (nginx couldn't read config files or wouldn't start at all).
- **C** is incorrect because if `defaultAction` were `SCMP_ACT_ALLOW`, everything would be allowed, and the profile would effectively be disabled—nginx would work.
- **D** is incorrect because Seccomp profiles don't conflict based on quantity; more allowed syscalls means less restriction, not more errors.

Troubleshooting Tip: Use `ausearch -m SECCOMP -ts recent` to check audit logs for blocked syscalls, or run with `SCMP_ACT_LOG` to identify missing syscalls without breaking functionality.

Learning Objective Tested: Debug and troubleshoot Seccomp-related application failures

Question 3: Seccomp Actions and Security

Question:

You are reviewing a Seccomp profile for a production Python application. The profile contains the following configuration:

```
{
  "defaultAction": "SCMP_ACT_LOG",
  "syscalls": [
    {
      "names": ["exit", "exit_group", "read", "write"],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

What is the security implication of this profile in a production environment?

- A)** The profile provides strong security by blocking all syscalls except the four explicitly allowed
- B)** The profile provides no actual security enforcement and is only suitable for development/testing
- C)** The profile will cause the application to crash when it attempts any syscall outside the allowed list
- D)** The profile is invalid and will be rejected by the container runtime

Correct Answer: B

Explanation:

The `defaultAction` is set to `SCMP_ACT_LOG`, which means that syscalls NOT in the allowed list will be permitted but logged to the audit system. This action is used for monitoring and development purposes to discover which syscalls an application uses without blocking them. In production, you should use `SCMP_ACT_ERRNO` (return error) or `SCMP_ACT_KILL` (terminate process) as the default action to actually enforce security restrictions.

Why other options are incorrect:

- **A** is incorrect because `SCMP_ACT_LOG` does not block syscalls; it only logs them while allowing them to execute.

- **C** is incorrect because the application will NOT crash— `SCMP_ACT_LOG` allows all syscalls to proceed.
- **D** is incorrect because this is a valid Seccomp profile format; `SCMP_ACT_LOG` is a legitimate action type.

Best Practice: Use `SCMP_ACT_LOG` in staging/development to build comprehensive profiles, then switch to `SCMP_ACT_ERRNO` for production enforcement.

Learning Objective Tested: Apply Seccomp profiles in production environments and understand different action types

Final Assessment Questions (2 MCQs - Comprehensive)

Question 4: Integrated Container Security Strategy

Question:

Your organization runs a microservices architecture on Kubernetes with strict security requirements. You've implemented the following controls:

1. Custom Seccomp profiles that allow only 50-60 syscalls per service
2. Read-only root filesystems for all containers
3. Pod Security Standards set to "Restricted"
4. Network policies limiting inter-pod communication

During a security audit, penetration testers successfully compromise a vulnerable web service container and gain root access inside the container. However, they report being unable to:

- Mount filesystems or access host devices
- Use kernel features like `bpf` or `keyctl`
- Modify files outside of designated writable volumes
- Communicate with pods outside the allowed network policy

Which layer of defense was MOST directly responsible for blocking the attempt to use kernel features like `bpf`?

- A)** Read-only root filesystem
- B)** Custom Seccomp profile
- C)** Pod Security Standards
- D)** Network policies

Correct Answer: B

Explanation:

The custom Seccomp profile is most directly responsible for blocking syscalls related to kernel features like `bpf` (Berkeley Packet Filter) and `keyctl` (kernel keyring management). Seccomp operates at the system call level, preventing processes from invoking specific kernel operations regardless of their privilege level. Since the Seccomp profile only allows 50-60 syscalls, advanced kernel features like `bpf` and `keyctl`—which are commonly used in container escape attempts—would be blocked at the syscall boundary.

Why other options are less directly responsible:

- **A** (Read-only root filesystem) prevents file modifications but does not block syscalls related to kernel features. An attacker could still attempt to call `bpf` or `keyctl` syscalls (they would be blocked by Seccomp, not by the read-only filesystem).

- **C** (Pod Security Standards) enforce security contexts and capabilities, which contribute to overall security but are enforced at the container runtime level, not at the syscall level. PSS “Restricted” would drop dangerous capabilities, but Seccomp provides the syscall-level enforcement.
- **D** (Network policies) control network traffic between pods, which is unrelated to kernel feature syscalls.

Defense in Depth Analysis:

This scenario demonstrates the principle of defense in depth:

- **Seccomp** blocks dangerous syscalls (mount, bpf, keyctl)
- **Read-only filesystem** prevents persistence and binary planting
- **Pod Security Standards** drop capabilities and enforce security contexts
- **Network policies** limit lateral movement

Even though one layer (the web application) was compromised, the other layers prevented further exploitation. This is exactly how layered security should work.

Learning Objective Tested: Design a layered container security architecture and understand how different controls complement each other

Question 5: Seccomp Troubleshooting in Production

Question:

Your team has deployed a containerized database application to production using a custom Seccomp profile. The application runs successfully for several days, but then starts failing intermittently with “Operation not permitted” errors during nightly backup jobs. The backup job involves creating a snapshot of the database and compressing it.

You check the audit logs and find repeated Seccomp denials for syscall 165 (`mount`) during the backup window. The backup script uses the following command:

```
tar -czf /backup/db-$(date +%Y%m%d).tar.gz /data
```

What is the MOST likely cause of this issue, and how should it be resolved?

- A)** The backup job requires mounting a filesystem; add the `mount` syscall to the Seccomp profile
- B)** The `tar` command in the container is trying to use advanced filesystem features; investigate why tar is calling `mount` and either fix the tar command or carefully evaluate if mount should be allowed
- C)** The Seccomp profile is corrupted; rebuild it from scratch using strace
- D)** The database has grown too large; increase container resource limits

Correct Answer: B

Explanation:

This scenario requires critical thinking. The `mount` syscall being called by a backup operation is highly unusual and potentially suspicious. The correct approach is to investigate WHY tar is attempting to mount filesystems, as this is not normal behavior for a simple tar compression operation.

Possible root causes:

1. The tar binary might be a malicious or modified version
2. The backup script might include additional commands not shown (like mounting network storage)

3. There could be a misconfiguration causing tar to access unusual filesystem features
4. The container might have been compromised and the “backup job” is actually an attack attempt

Correct resolution approach:

1. **Investigate the tar binary:** Check its integrity and compare with known-good versions
2. **Review the complete backup script:** There may be additional commands that require mount
3. **Examine the /backup mount point:** Determine if it’s a network mount (NFS, CIFS) that requires the mount syscall
4. **Consider alternatives:** Use volume mounts configured at container start rather than dynamic mounts

Why other options are incorrect:

- **A** is incorrect and dangerous because blindly adding `mount` to the Seccomp profile reopens a critical attack vector. The `mount` syscall is blocked by default for good reason—it can be used to access host filesystems and escape the container.
- **C** is incorrect because the profile is working correctly (it’s blocking mount as intended); rebuilding it won’t change the fact that tar is trying to call mount.
- **D** is incorrect because resource limits are unrelated to syscall permissions; database size doesn’t cause Seccomp denials.

Security Principle:

This question tests the understanding that **not all Seccomp denials should be resolved by adding the blocked syscall**. Some denials indicate suspicious behavior that requires investigation. Always ask “WHY is this syscall being called?” before allowing it, especially for high-risk syscalls like `mount`, `ptrace`, `keyctl`, or `bpf`.

Best Practice Response:

1. Investigate the root cause
2. If mount is legitimately needed (e.g., for NFS backup storage), use host-level mounts configured in the container/pod spec instead of allowing the syscall
3. Consider using an init container or sidecar for backup operations with a separate, slightly less restrictive profile
4. Document the investigation and decision in your security documentation

Learning Objective Tested:

- Troubleshoot Seccomp denials with critical thinking
 - Understand security implications of allowing specific syscalls
 - Balance security requirements with operational needs
 - Recognize that not all “Operation not permitted” errors should be fixed by allowing the operation
-

Answer Key Summary

Question	Correct Answer	Difficulty	Topic
Q1	B	Easy	Seccomp Fundamentals
Q2	B	Medium	Profile Creation & Debugging
Q3	B	Medium	Seccomp Actions
Q4	B	Hard	Integrated Security Strategy
Q5	B	Hard	Production Troubleshooting

Assessment Guidelines

Module-Level Questions (Q1-Q3):

- **Purpose:** Verify understanding of core concepts from Module 3
- **Passing Criteria:** 2 out of 3 correct (66%)
- **Time Limit:** 5 minutes
- **Retake Policy:** Unlimited retakes with randomized options

Final Assessment Questions (Q4-Q5):

- **Purpose:** Test ability to apply knowledge across multiple security domains
- **Passing Criteria:** Both questions correct for full marks; part of 10-question final exam
- **Time Limit:** Part of 20-minute final assessment
- **Retake Policy:** One retake after reviewing all course materials

Question Difficulty Calibration:

- **Easy (Q1):** Direct recall of concepts from video lectures
- **Medium (Q2-Q3):** Application of concepts to realistic scenarios
- **Hard (Q4-Q5):** Synthesis across multiple topics, critical thinking, and security judgment

Instructor Notes

Common Student Mistakes:

Q1: Students may confuse Seccomp with network isolation or image scanning. Emphasize that Seccomp is specifically about syscall filtering.

Q2: Students may select option A because `read` and `write` seem fundamental. Stress that nginx startup vs. runtime behavior requires different syscalls.

Q3: Students may think `SCMP_ACT_LOG` provides security because it has “seccomp” in the name. Clarify the difference between monitoring and enforcement actions.

Q4: Students may select C (Pod Security Standards) because it sounds comprehensive. Explain that PSS works at a higher level than syscall filtering.

Q5: This question intentionally mimics real production scenarios where the “obvious” answer (add the syscall) is wrong. Many students will choose A. Use this to teach security judgment.

Discussion Topics (if using questions in live training):

- **After Q2:** Discuss real-world debugging workflows and tools beyond those in the question
- **After Q3:** Show examples of profiles in different stages (development vs production)
- **After Q4:** Analyze a real container breach and map it to the defense layers
- **After Q5:** Facilitate group discussion on security vs. operational trade-offs

Adaptive Learning Recommendations:

If student misses Q1:

- Review Video 1, especially the section on syscall filtering
- Complete Lab 3.1 with detailed observation of blocked vs. allowed syscalls

If student misses Q2:

- Review Video 2, focusing on the strace workflow
- Complete Lab 3.2 with emphasis on iterative testing
- Practice with the broken profile debugging exercise

If student misses Q3:

- Review the Seccomp actions table in course materials
- Review Video 3 section on permissive mode vs. enforcement
- Compare `SCMP_ACT_LOG`, `SCMP_ACT_ERRNO`, and `SCMP_ACT_KILL` in practice

If student misses Q4:

- Review the defense-in-depth section across all modules
- Complete the integrated security lab (capstone)
- Study the security layers diagram

If student misses Q5:

- Review Video 3 troubleshooting section
- Discuss with instructor about security decision-making
- Complete advanced troubleshooting lab with mentor guidance
- Review case studies of production Seccomp incidents